Name:

PRN:

Experiment No. 7

Title: MPI Collective Communication and Parallel Matrix Computations

1. Aim

To understand and implement MPI collective communication operations and apply them to parallel matrix computations using distributed-memory programming.

You will:

- Understand collective communication in MPI.
- Implement MPI_Bcast(), MPI_Scatter(), MPI_Gather(), and MPI_Reduce().
- Implement parallel Matrix–Vector Multiplication using MPI.
- Implement parallel Matrix–Matrix Multiplication using MPI.
- Apply different data decomposition strategies.
- Measure and analyze communication and computation performance.
- Understand the effect of the number of processes and decomposition strategy on performance.

2. Why Are We Learning This?

In distributed-memory HPC systems, data is divided among multiple processes. Unlike OpenMP, where threads share a common memory, MPI processes need to exchange data explicitly.

For example, consider a large matrix:

[C = A* B]

If the matrix is too large for a single machine or the computation is time-consuming, different portions of the matrix can be assigned to different processes.

MPI collective operations provide efficient mechanisms to distribute and combine this data.

These concepts are fundamental to applications such as:

- Scientific and engineering simulations
- Computational fluid dynamics
- Weather and climate modelling

- Image and video processing
- Machine learning and AI
- Large-scale numerical linear algebra
- Distributed data analytics

3. HPC in Practice

Consider a weather simulation working with a large numerical grid.

The complete grid cannot efficiently be processed by one process. The grid can be divided among multiple processes:

Process 0 → Region 1
Process 1 → Region 2
Process 2 → Region 3
Process 3 → Region 4

MPI can then be used to:

1. Distribute input data.
2. Perform computation on local data.
3. Exchange required information.
4. Collect the final results.

This pattern of:

Distribute → Compute → Communicate → Combine

is one of the fundamental patterns in HPC programming.

4. Learning Focus

Concept	MPI Function / Technique
Broadcast data	MPI_Bcast()
Distribute data	MPI_Scatter()
Collect data	MPI_Gather()
Combine results	MPI_Reduce()
Synchronization	MPI_Barrier()
Matrix–Vector Multiplication	Row-wise decomposition
Matrix–Matrix Multiplication	Row-wise / block decomposition
Performance	Time, speedup, efficiency

Concept	MPI Function / Technique
Advanced concept	MPI_Scatterv()

5. Pre-Lab Activity

Before performing the assignment, answer the following:

1. What is collective communication in MPI?
2. How is collective communication different from point-to-point communication?
3. What is the purpose of MPI_Bcast()?
4. What is the difference between MPI_Scatter() and MPI_Gather()?
5. What does MPI_Reduce() do?
6. Why is MPI_Barrier() used?
7. What is data decomposition?
8. Why is matrix computation suitable for parallel processing?
9. What is row-wise decomposition?
10. What happens if the number of matrix rows is not evenly divisible by the number of processes?

Section A – Basics: MPI Collective Communication

Task 1: Broadcast Using MPI_Bcast()

Write an MPI program in which:

- Process 0 initializes an integer value.
- Process 0 broadcasts the value to all other processes using MPI_Bcast().
- Every process displays the received value and its rank.

Suggested execution

Run the program using:

- 2 processes
- 4 processes
- 8 processes

Observe

Create a table:

Number of Processes	Process Rank	Received Value
4	0	
4	1	
4	2	
4	3	

Think

Why is broadcasting more appropriate than performing multiple individual MPI_Send() operations?

Task 2: Scatter and Gather

Create an integer array of size 16 at Process 0.

Use:

MPI_Scatter()

to distribute the array among all processes.

Each process should:

1. Receive its portion.
2. Calculate the local sum.
3. Send the local result back using MPI_Gather().

Display the local sum calculated by each process and the final result at Process 0.

Example

For:

Array = [1,2,3,...,16]

with 4 processes:

Process 0 → [1,2,3,4]
Process 1 → [5,6,7,8]
Process 2 → [9,10,11,12]
Process 3 → [13,14,15,16]

Task 3: Global Reduction Using MPI_Reduce()

Modify Task 2 to use:

MPI_Reduce()

to calculate the global sum.

Also calculate:

- Global maximum
- Global minimum

using appropriate MPI reduction operations.

Compare the use of MPI_Gather() and MPI_Reduce().

Discussion

Why is MPI_Reduce() preferable when only the final aggregate value is required?

Section B – Moderate Application: Matrix Computations

Task 4: MPI Matrix–Vector Multiplication

Implement:

[Y = A X]

where:

- (A) is an (N N) matrix.
- (X) is an (N) vector.
- (Y) is an (N) result vector.

Step 1 – Sequential Version

Implement the matrix-vector multiplication sequentially.

Measure its execution time.

Step 2 – MPI Version

Implement the MPI version using row-wise decomposition.

Vector X → Broadcast to all processes

Use:

- MPI_Scatter() to distribute matrix rows.
- MPI_Bcast() to distribute the vector.
- Local computation at each process.
- MPI_Gather() to collect the result.

Performance Analysis

Test with different matrix sizes, for example:

- 500 × 500
- 1000 × 1000
- 2000 × 2000

and different numbers of processes:

- 2
- 4
- 8

Record:

Matrix Size	Processes	Sequential Time	MPI Time	Speedup	Efficiency
500	2				
500	4				
500	8				
1000	2				
1000	4				
1000	8				

Task 5: MPI Matrix–Matrix Multiplication

Implement:

[C = A * B]

for two square matrices (A) and (B).

Sequential Version

First implement the sequential algorithm.

MPI Version

Implement a parallel version using row-wise decomposition.

Suggested approach:

1. Process 0 initializes matrices (A) and (B).
2. Distribute rows of (A) using MPI_Scatter().
3. Broadcast matrix (B) using MPI_Bcast().
4. Each process computes its assigned rows of (C).
5. Collect the rows of (C) using MPI_Gather().

Section C – Challenge: Decomposition and Optimization

Task 6: Compare Different Matrix Decomposition Strategies

Modify the MPI matrix-matrix multiplication program to investigate different decomposition strategies.

Study:

1. Row-wise decomposition

Each process receives a set of rows of matrix (A).

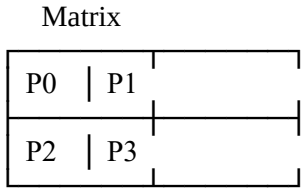
2. Column-wise decomposition

Each process is responsible for a set of columns of matrix (B).

3. Block decomposition

Divide the matrix into smaller blocks.

For example:



Investigate how the decomposition affects:

- Amount of data communicated.

- Computation performed by each process.
- Memory usage.
- Load balancing.
- Execution time.
- Scalability.

Task 7: Non-Uniform Data Distribution Using MPI_Scatterv()

Consider a situation where the matrix cannot be divided equally among the processes.

For example:

N = 10
Number of processes = 3

The rows cannot be divided equally.

Investigate and implement matrix-vector multiplication using:

MPI_Scatterv()

instead of MPI_Scatter().

The distribution could be:

P0 → 4 rows
P1 → 3 rows
P2 → 3 rows

Analyze

Compare:

MPI_Scatter()
vs.
MPI_Scatterv()

in terms of:

- Data distribution
- Flexibility
- Programming complexity
- Applicability to real HPC problems

6. Performance Analysis

For your MPI programs, measure:

Execution Time

Speedup

Efficiency

where (p) is the number of processes.

Also investigate:

- Communication overhead
- Computation time
- Load imbalance
- Scalability
- Effect of matrix size
- Effect of number of processes

Required Graphs

Prepare:

1. **Number of Processes vs Execution Time**
2. **Number of Processes vs Speedup**
3. **Number of Processes vs Efficiency**
4. **Matrix Size vs Execution Time**

7. AI/HPC Case Study

Select one real-world application where distributed matrix computation or collective communication is used.

Suggested areas:

- Deep learning
- Computer vision
- Scientific computing
- Weather forecasting
- Computational fluid dynamics
- Medical imaging
- Recommendation systems
- Large-scale data analytics

- Molecular simulation

Write **300–500 words** covering:

1. Problem being solved.
2. Why the computation is computationally intensive.
3. How data can be distributed.
4. Role of MPI communication.
5. Possible decomposition strategy.
6. Benefits of parallel execution.
7. Communication or scalability challenges.

8. Discussion Questions

1. What is collective communication?
2. How does MPI_Bcast() work?
3. Differentiate MPI_Scatter() and MPI_Gather().
4. Differentiate MPI_Gather() and MPI_Reduce().
5. Why is MPI_Bcast() useful in matrix multiplication?
6. Why is row-wise decomposition suitable for matrix-vector multiplication?
7. What are the limitations of row-wise decomposition?
8. What is block decomposition?
9. Why can block decomposition improve data locality?
10. What is the purpose of MPI_Scatterv()?
11. What happens when the number of data elements is not evenly divisible by the number of processes?
12. Why does execution time sometimes stop decreasing as the number of processes increases?
13. How does communication overhead affect MPI scalability?
14. How does MPI differ from OpenMP in terms of memory architecture?
15. For a very large matrix, what factors should be considered when selecting a decomposition strategy?

9. Reflection

Answer the following in your own words:

1. What was the most important concept you learned from this assignment?
2. Which MPI collective operation did you find most useful and why?
3. How is MPI matrix multiplication different from OpenMP matrix multiplication?
4. What happens when more MPI processes are added?

5. Did increasing the number of processes always improve performance? Explain using your observations.
6. Which decomposition strategy would you select for a large matrix and why?
7. What communication bottlenecks did you observe?
8. How could the implementation be further optimized for an HPC cluster?

10. Deliverables

Submit the following:

A. Programs

1. MPI Broadcast
2. MPI Scatter/Gather
3. MPI Reduce
4. MPI Matrix–Vector Multiplication
5. MPI Matrix–Matrix Multiplication
6. Decomposition comparison
7. MPI_Scatterv() implementation

B. Documentation

Include:

- Aim and objective
- Algorithm
- Source code
- Sample input/output
- Performance tables
- Graphs
- Speedup calculation
- Efficiency calculation
- Analysis of results
- AI/HPC case study
- Discussion-question answers
- Reflection
- Conclusion

C. Performance Evidence

Include execution results for different:

- Matrix sizes
- Number of MPI processes

- Decomposition strategies

11. Expected Learning Outcomes

After completing this assignment, you should be able to:

1. **Implement** MPI collective communication operations such as broadcast, scatter, gather and reduction.
2. **Apply** MPI to matrix-vector and matrix-matrix computations.
3. **Analyze** different data decomposition strategies for distributed-memory systems.
4. **Measure** execution time, speedup, efficiency and communication overhead.
5. **Evaluate** the effect of process count and data distribution on MPI performance.
6. **Relate** MPI-based parallel computation to real-world HPC and AI applications.